Tractable Problems in AI Security via Formal Methods

A sane way to burn a few postdoc-years

06.17.2026

Quinn Dougherty

Contents

1 The strategic landscape

Let us treat ML training and inference with the seriousness of airplanes

Authors

Quinn Dougherty, Max von Hippel, Gregory Malecha, Nora Ammann

We mostly write about model weight confidentiality and integrity, but other notions of AI security are in scope. Not the biggest possible tent, but more than two invariants.

Figure 1: Demands hardened critical infrastructure, doesn't say how.

Figure 2: Expert survey, human and simulated/silicon

ARIA

TODO

Figure 3: See *Leveling up across the board* at https://www.anthropic.com/responsible-scaling-policy/roadmap

Speed

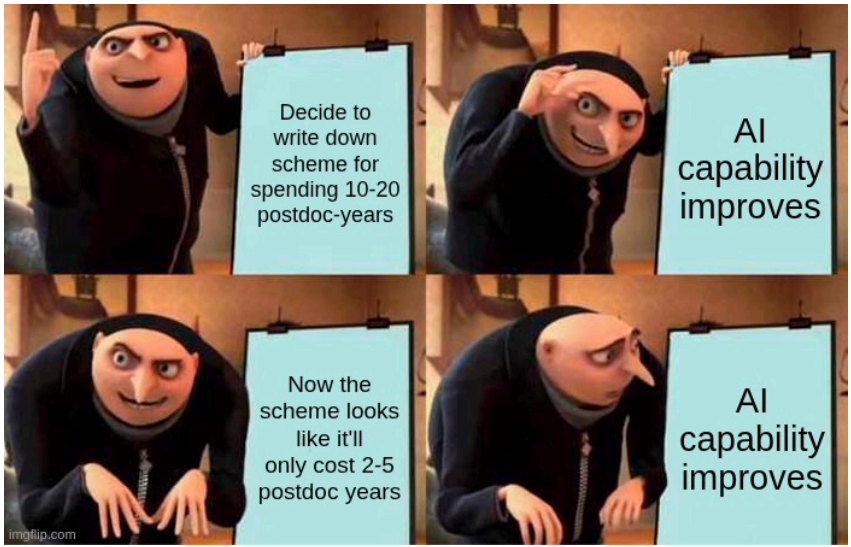

Figure 4: This is technically a violation of the meme format.

2 The stack

5 layers

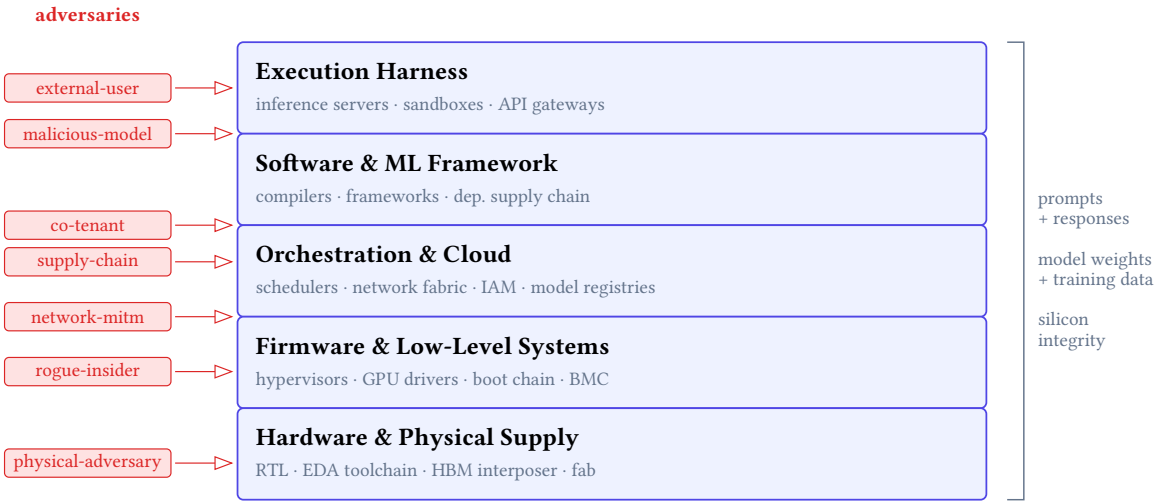

Figure 5: The path of a prompt from the enduser to the chip's circuits

3 Tractable Problems

Hardening protocol boundaries

Chapter with Nora Ammann

Before any AI-specific machinery runs, a request has already crossed TLS, SSH, and WireGuard. The oldest trust boundaries, still the largest single-bug blast radius.

Status quo

Heartbleed, FREAK, Triple Handshake on TLS; `regreSSHion` (pre-auth remote root) on SSH — a signal-handler race, not even a parser bug. ARIA names verified TLS 1.3, SSH, and WireGuard as three of six example targets.

The shared shape

Every catastrophic bug is one of two things — memory unsafety in the parser, or an unintended transition in the state machine (downgrade, pre-auth confusion, stale-credential reuse). One LangSec pattern across all three protocols.

Hardening protocol boundaries (ii)

The good news

Each piece is already tractable in isolation — `EverParse` (verified parsers), Project Everest / miTLS (TLS in `F*`), Owl, `Tamarin`/`ProVerif`. Assume the crypto primitives (HACL*/EverCrypt); the kernel and side channels are out of scope.

The real blocker

No one ships a **single** proof spanning the whole boundary — parser, both state machines, and the daemon's concurrency/signal discipline — as a drop-in at a real edge. That gap, not a missing technique, is the widget.

The widget

Three instances sharing a parser-plus-state-machine method, so 2 and 3 are cheaper than 1: a drop-in TLS 1.3 server, a verified SSH boundary (covering the concurrency `regreSSHion` exploited), and a verified WireGuard control plane (tunnel assumed verified; prove the membership state machine).

Hardening OCI runtime

Chapter with Max von Hippel

The container is the agent's cage. `runc` was never built to hold an adversary *inside* it.

Status quo

The standard runtime (`runc`) implements OCI atop Linux namespaces and cgroups — process isolation, never a security boundary. With an agent that writes and runs arbitrary code, that gap is the whole game.

The good news

We can now *measure* the gap. Frontier models are red-teamed for sandbox escape directly (UK AISI's toolkit + Docker breakout CTF, BashArena); BoxArena flips it to a **defensive** leaderboard — fix the attacker, vary the runtime (`runc`, `gVisor`, `crun`, `Kata`) across five surfaces.

Hardening OCI runtime (ii)

The real blocker

Empirical red-teaming is a treadmill — find a hole, patch it, a new escape surfaces next quarter. A capable enough agent should be assumed to find any flaw in its containment. That's the cue for proof, not more benchmarks.

The widget

A verified OCI runtime. Model the 80 syscalls a hardened container needs, specify the confinement policy (drop caps, read-only rootfs, `seccomp` allowlist, no-new-privs), prove no syscall sequence from inside breaks it. Pair the verified boundary with a scalable monitor — each makes the other's job easier.

Device drivers for verified kernels

Chapter with Gregory Malecha

The kernel can be verified; the driver underneath it usually can't.

Status quo

Multi-tenant GPU isolation rides on million-line hypervisor TCBs, with a proprietary ring-0 GPU driver as the single point of compromise.

The good news

`seL4`, `NOVA`, `seKVM`, and AWS Nitro verify a minimal core and push drivers out to deprivileged user-mode — the right architecture.

Device drivers for verified kernels (ii)

The real blocker

Not the proof, the spec. Driver proofs are a settled methodology (a verified ZynqMP DMA engine exists); what's missing is a machine-checkable model of the GPU's command / DMA / IOMMU surface. Vendors model the **ISA** — Arm, RISC-V, even AMD's shaders — not the device.

The widget

Specify one open GPU stack's command-submission ring (`NVK`/`Nouveau`) in `Rocq`/`Lean`; prove no guest command can drive a DMA or IOMMU mapping outside its region. Stub the hardware model first (reference), then co-develop a real one (research).

4 Onwards

How you can help

More contributions

Come aboard! More room for authors. Or, if you want to be low-key, comment on the website's native comments feature.

Tokens

Ask claude to implement one of the project sketches on `/problems`. Let us know how it goes.

Podcasts

What podcasts should we go on to discuss this? Get us invited.

I'm easy to find

`quinn@for-all.dev`